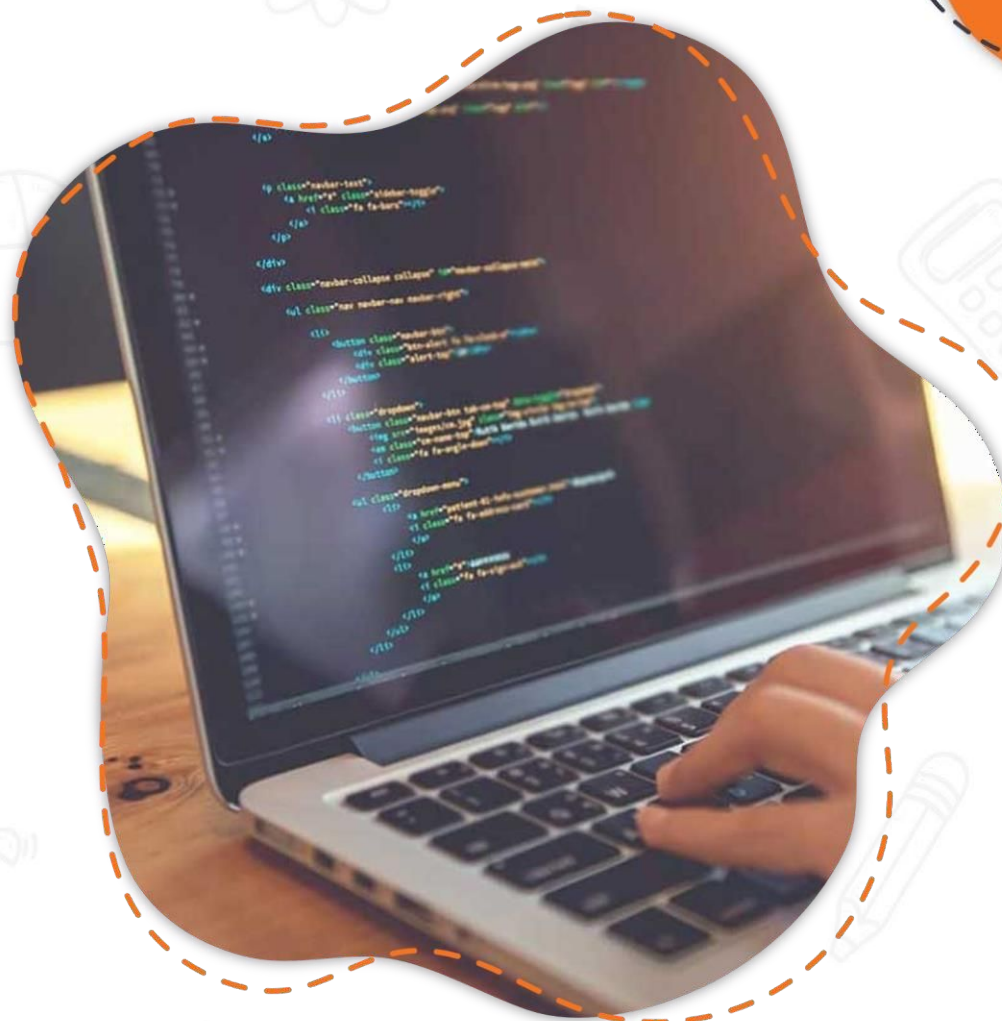


Java Backend



1.

Kiến trúc servlet

2.

Http Servlet

3.

Tạo project servlet

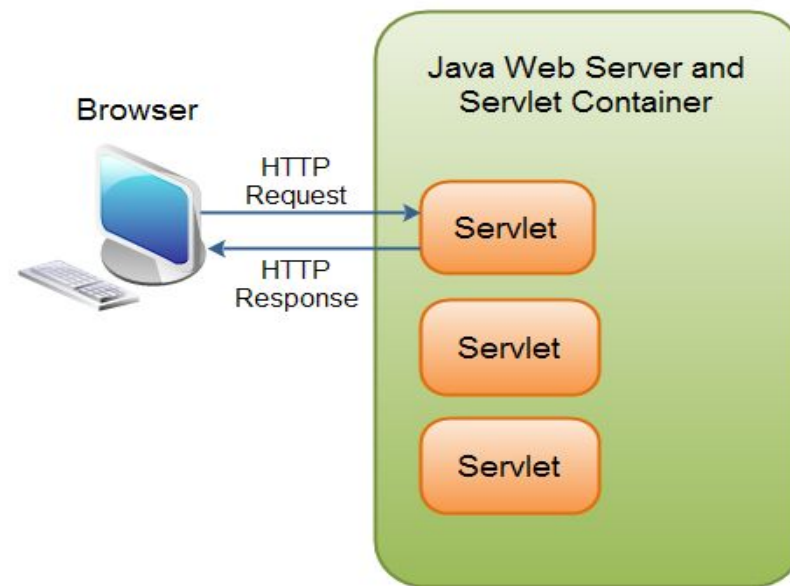
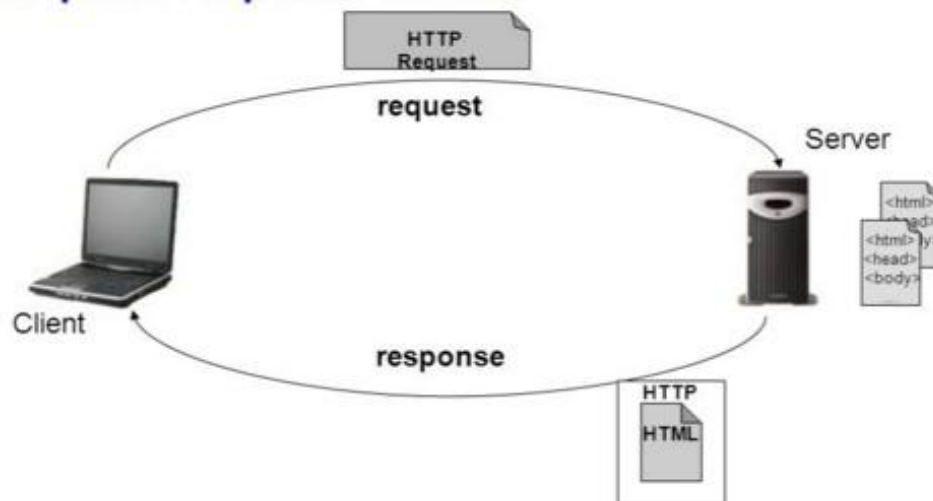
4.

Servlet Request and Response

01 Kiến trúc Servlet

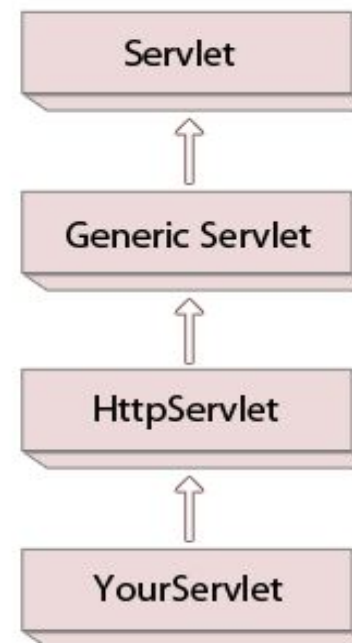
- ❑ Servlet là một thành phần phía server trong Java, được sử dụng để xử lý các yêu cầu (request) từ client (thường là trình duyệt web) và tạo ra phản hồi(response) động. Nó chạy bên trong một **Web container** (ví dụ: Apache Tomcat, Jetty).

- Request-response model.



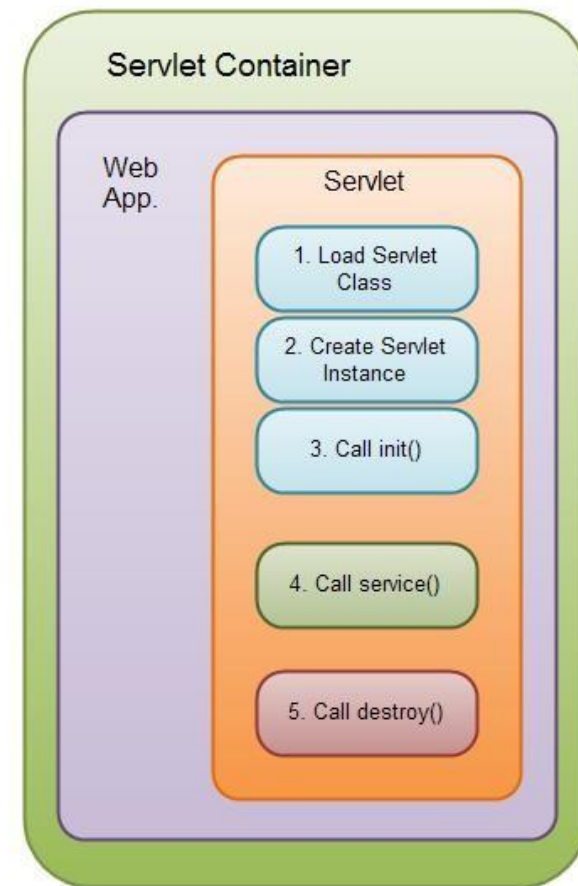
01 Kiến trúc Servlet

- ❑ Để tạo 1 servlet ta sẽ tạo class implements interface **javax.servlet.Servlet**.
 - ❖ Class có thể implement trực tiếp Servlet interface hoặc kế thừa gián tiếp thông qua GenericServlet hoặc HttpServlet.
 - ❖ Mục đích việc kế thừa HttpServlet class để có thể sử dụng HTTP services cho class servlet.



01**Vòng đời Servlet****❑ Bao gồm 5 bước.**

- ❖ Bước 1: Tải lớp Servlet vào bộ nhớ.
- ❖ Bước 2: Tạo đối tượng Servlet.
- ❖ Bước 3: Gọi phương thức `init()` của Servlet.
- ❖ Bước 4: Gọi phương thức `service()` của Servlet.
- ❖ Bước 5: Gọi phương thức `destroy()` của Servlet.

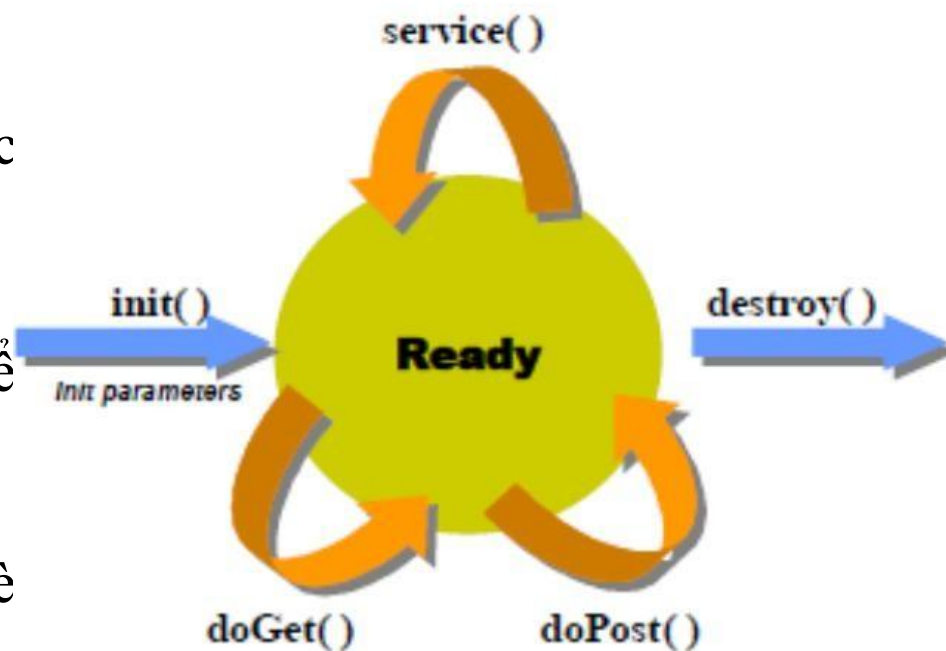


01 Vòng đời Servlet

- ☐ **Bước 1, 2 và 3** được thực thi một lần duy nhất, khi mà Servlet được nạp lần đầu. Mặc định các Servlet không được tải (load) lên cho tới khi nó nhận một đòi hỏi đầu tiên từ người dùng. Bạn có thể bắt buộc Servlet Container (Bộ chứa các Servlet) tải các Servlet khi nó khởi động.
- ☐ **Bước 4** được thực thi nhiều lần, mỗi khi có đòi hỏi từ phía người dùng tới Servlet.
- ☐ **Bước 5** được thực thi khi bộ chứa Servlet (Servlet Container) gỡ bỏ tải (unloaded) một Servlet.

01 Vòng đời Servlet

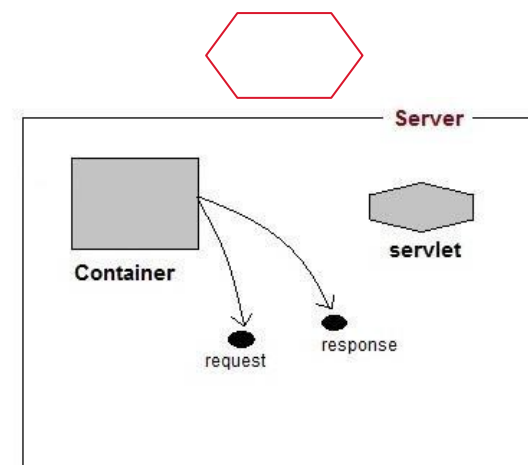
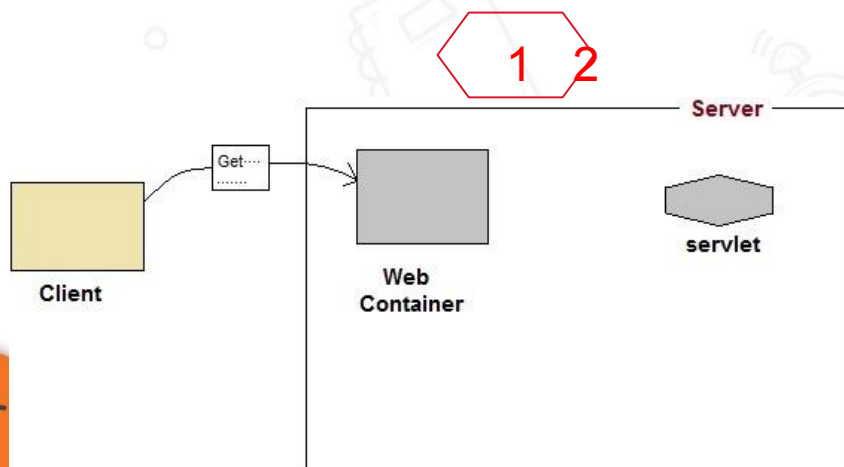
- ❑ Khi người dùng gửi yêu cầu một Servlet, Servlet sẽ được tạo ra tại thời điểm có yêu cầu lần đầu tiên tới, đồng thời sẽ gọi phương thức **init()** của servlet để khởi tạo cho nó, **init()** được gọi duy nhất 1 lần.
- ❑ Khi yêu cầu (request) của người dùng gửi tới Servlet, Servlet sẽ gọi phương thức **service()** để phục vụ yêu cầu của người dùng, **service()** sẽ gọi một trong hai phương thức **doGet()** hoặc **doPost()**. Trong Servlet của bạn, bạn cần ghi đè và xử lý tại các phương thức này



01

Vòng đời Servlet(Web container)

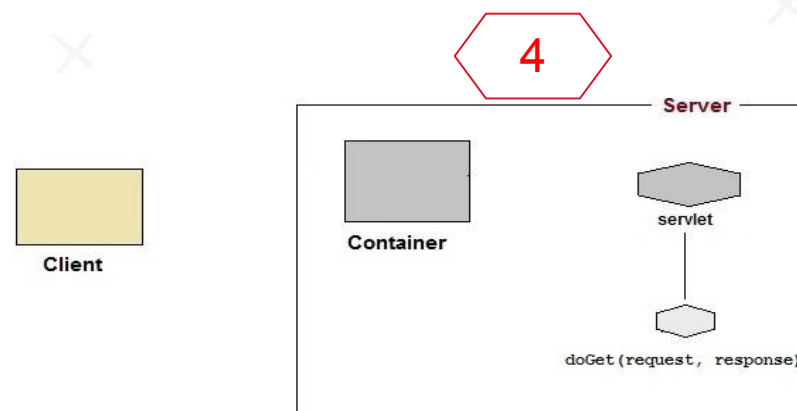
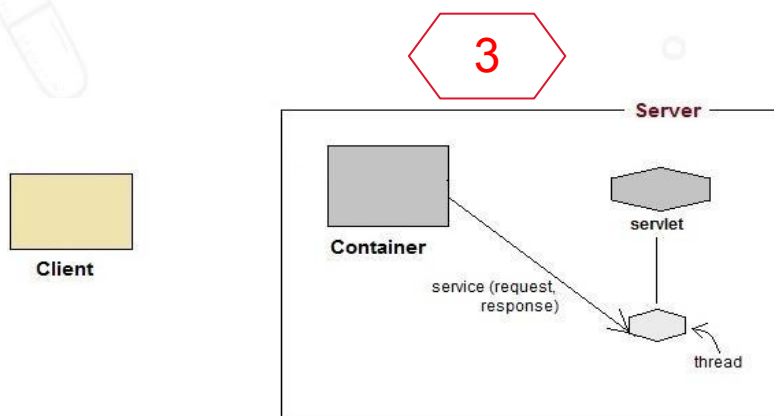
- ❑ **Web Container** Web Container (như Apache Tomcat, Jetty, hoặc GlassFish) là môi trường chạy cho các Servlet. Nó quản lý vòng đời của Servlet. Xử lý các yêu cầu từ client và chuyển tiếp chúng đến Servlet thích hợp. Đảm bảo tính bảo mật, hiệu suất và khả năng mở rộng của ứng dụng web.
- ❑ Khi người dùng request lên servlet, container sẽ tìm kiếm servlet (sử dụng **deployment descriptor**) và tạo 2 object **HttpServletRequest** và **HttpServletResponse**



01

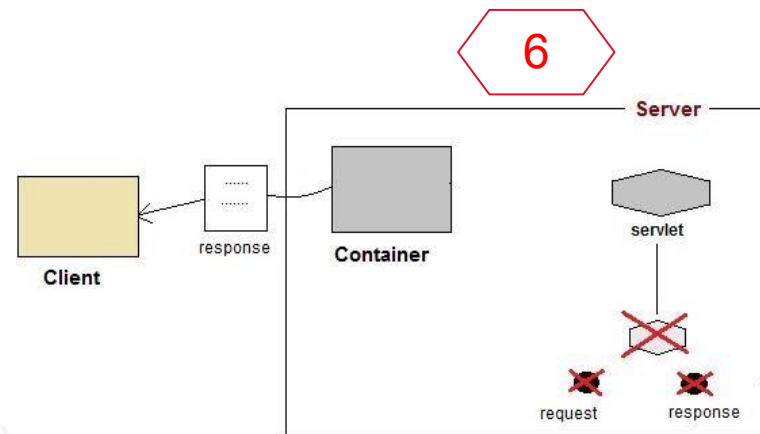
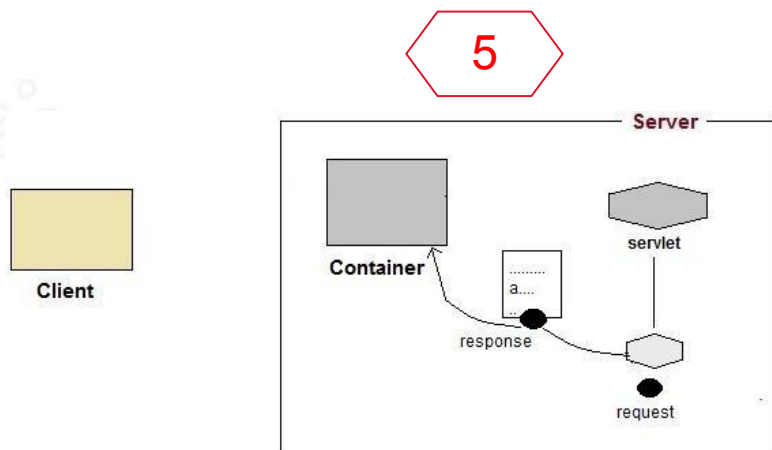
Vòng đời Servlet(Web container)

- ❑ Sau đó container gọi hàm **service()** của servlet là truyền tham số đầu vào là request response
- ❑ Phương thức **service()** sẽ quyết định gọi đến phương thức **doGet()** và **doPost()**



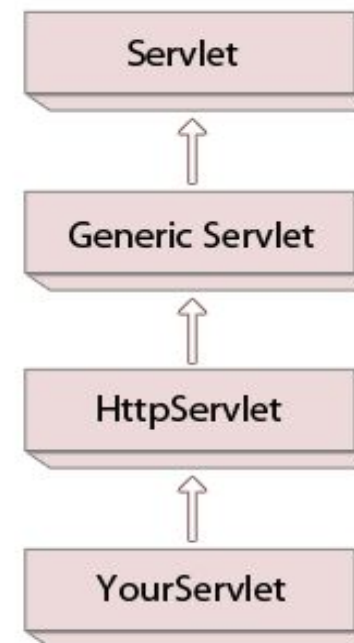
01 Vòng đời Servlet(Web container)

- ❑ Cuối cùng servlet truyền kết quả trả về cho khách hàng (client) thông qua HttpServletResponse



02 HttpServlet

- ❑ Class HttpServlet Kế thừa GenericServlet và implements Serializable interface
 - ❖ Cung cấp các phương thức HTTP doGet, doPost, doHead, doTrace
 - ❖ Phương thức GET trở đến phương thức doGet()
 - ❖ Phương thức POST trở đến phương thức doPost()
 - ❖ **HttpServletRequest** chứa thông tin request từ client,
 - ❖ **HttpServletResponse** cung cấp các chức năng để trả về dữ liệu text, html dữ liệu binary như image (HTTP headers, cookies...)



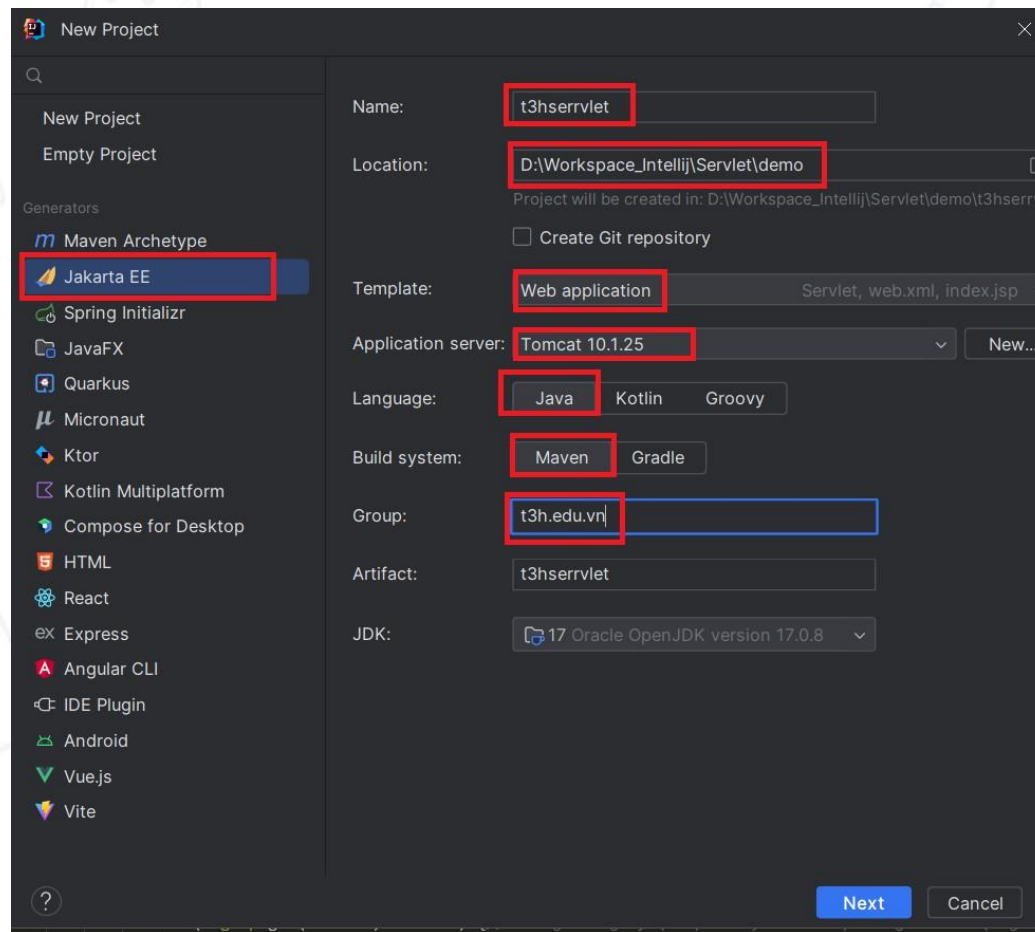
02 HttpServlet

- ❑ `public void service(ServletRequest req, ServletResponse res):`
 - ❖ Chuyển tiếp request đến phương thức protected service và chuyển đổi request và response sang httpRequest và httpResponse.
- ❑ `protected void service(HttpServletRequest req, HttpServletResponse res):`
 - ❖ Nhận request từ phương thức service và chuyển tiếp tới phương thức doXXX()
- ❑ `protected void doGet(HttpServletRequest req, HttpServletResponse res):`
- ❑ `protected void doPost(HttpServletRequest req, HttpServletResponse res):`
- ❑ `protected void doHead(HttpServletRequest req, HttpServletResponse res)`
- ❑ `protected void doOptions(HttpServletRequest req, HttpServletResponse res)`
- ❑ `protected void doPut(HttpServletRequest req, HttpServletResponse res)`
- ❑ `protected void doDelete(HttpServletRequest req, HttpServletResponse res)`
 - ❖ Phương thức GET gọi doGet(), POST gọi doPost(), HEAD gọi doHead(), OPTIONS gọi doOptions(), PUT gọi doPut(), DELETE gọi doDelete()...

03

Tạo servlet bằng IDE

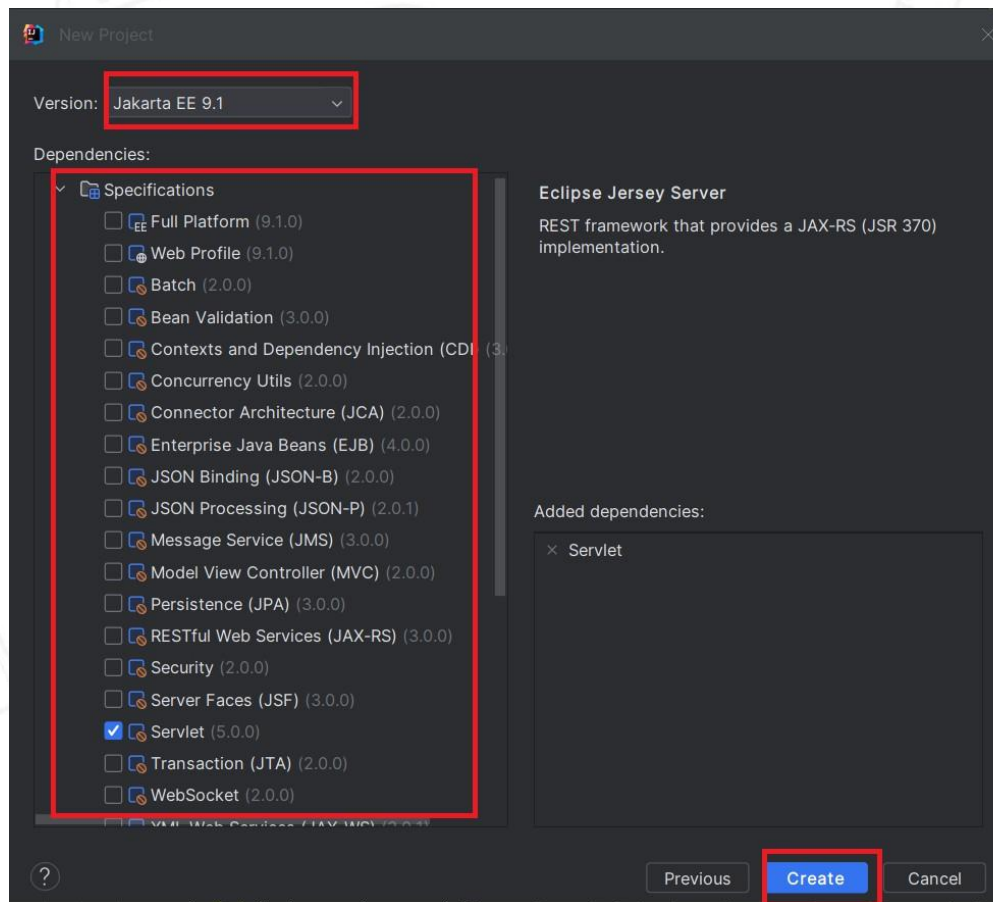
☐ Tạo mới Project



03

Tạo servlet bằng IDE

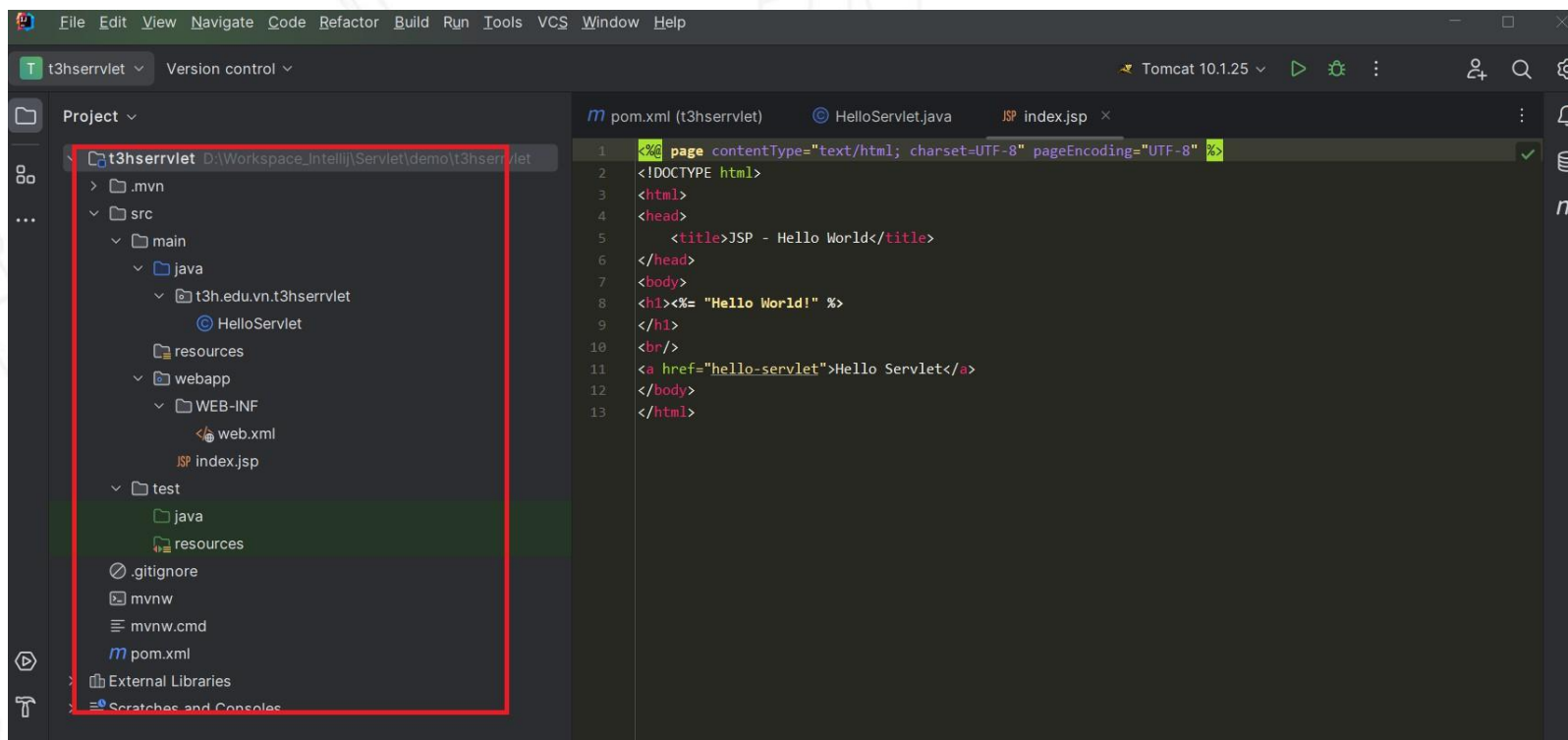
☐ Tạo mới Project



03

Tạo servlet bằng IDE

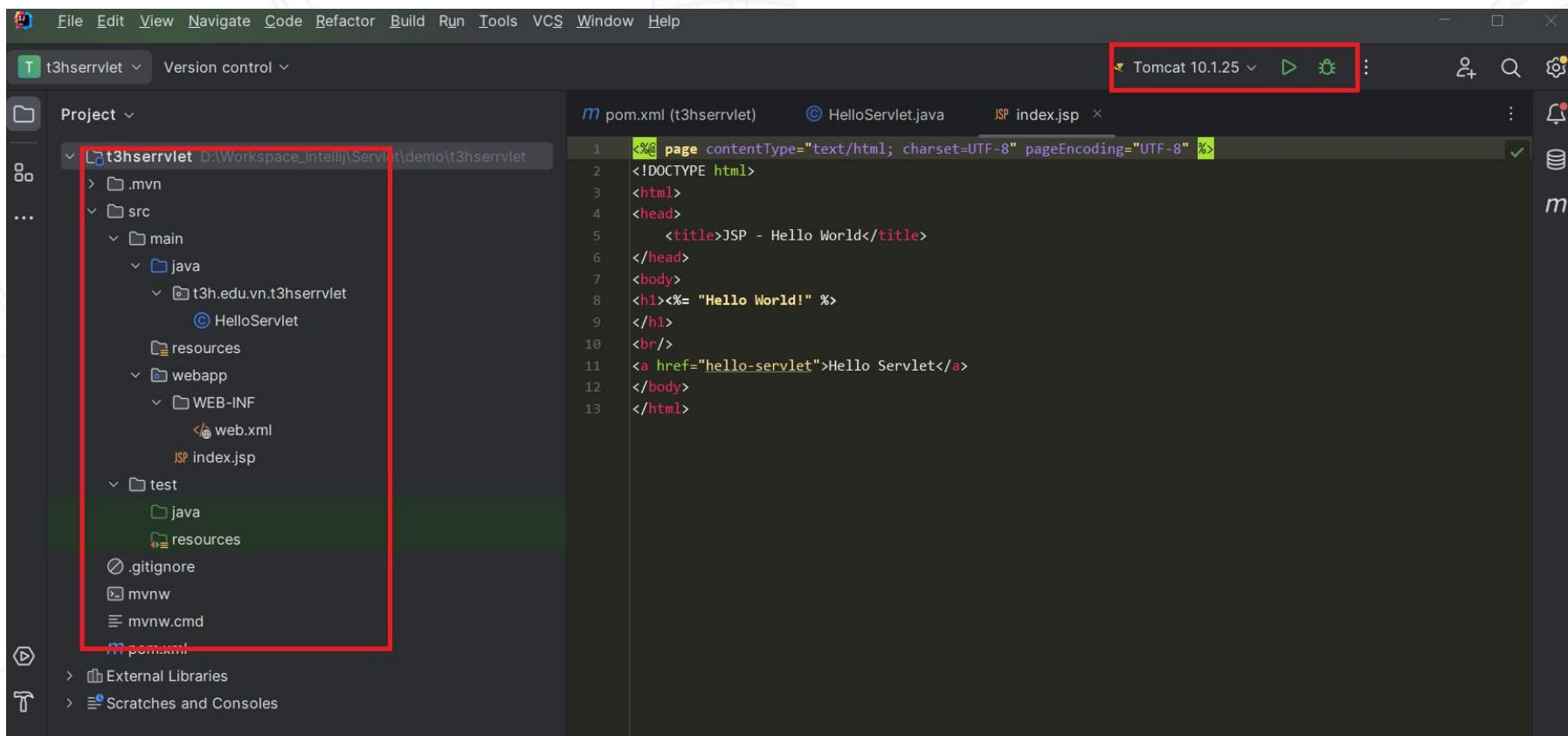
☐ Thành phần Project



03

Tạo servlet bằng IDE

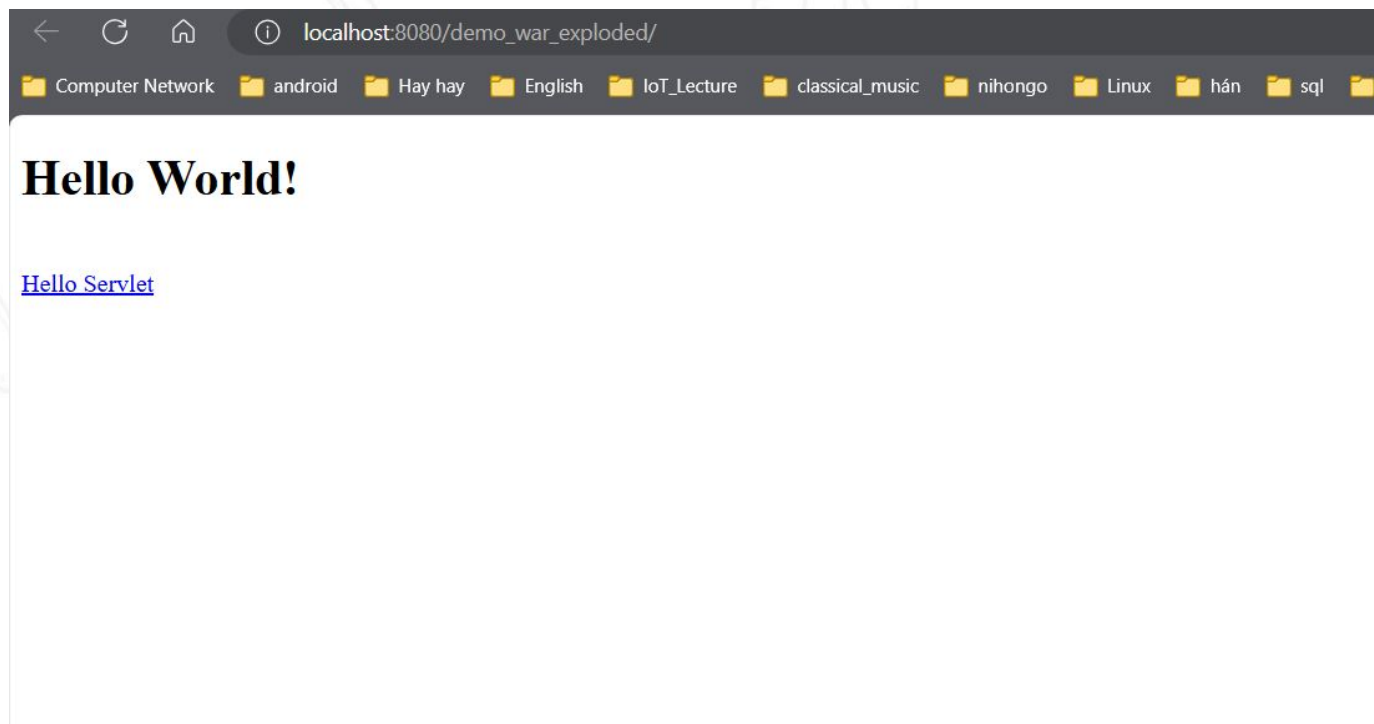
☐ Thành phần Project



03

Tạo servlet bằng IDE

☐ Kết quả



04 Servlet Request and Response

- ❑ **HttpServletRequest** là một giao diện (interface) trong Java Servlet API, được sử dụng để cung cấp thông tin chi tiết về yêu cầu HTTP từ client (chẳng hạn như trình duyệt web) đến server.
- ❑ Đây là một trong những đối tượng quan trọng trong Java Servlet, được Web Container truyền đến phương thức xử lý yêu cầu (như doGet, doPost) của Servlet.

04 Chức năng của Http Servlet Request

- **Truy xuất thông tin yêu cầu HTTP:**
 - Thông tin về phương thức HTTP (GET, POST, PUT, DELETE, v.v.).
 - Đường dẫn URL mà client gửi yêu cầu.
 - Tham số (parameters) được gửi trong URL hoặc trong phần thân (body) của yêu cầu.
- **Quản lý phiên làm việc (Session):**
 - Truy xuất hoặc tạo mới session thông qua phương thức getSession().
- **Truy cập header HTTP:**
 - Lấy các thông tin như loại nội dung (Content-Type), chiều dài nội dung (Content-Length), thông tin user-agent, cookie,...
- **Xử lý dữ liệu từ form HTML:**
 - Thu thập giá trị của các input thông qua các tham số như getParameter(String name).

04 Chức năng của Http Servlet Request

```
@WebServlet("/demo-request") new *
public class HttpServletRequestDemo extends HttpServlet {
    @Override no usages new *
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Thiết lập nội dung phản hồi dạng HTML
        response.setContentType("text/html");
        response.setCharacterEncoding("UTF-8");
        // 1. Truy xuất thông tin yêu cầu HTTP
        String method = request.getMethod(); // Phương thức HTTP (GET, POST, ...)
        String requestURL = request.getRequestURL().toString(); // URL đầy đủ
        String queryString = request.getQueryString(); // Chuỗi truy vấn (query string)
        String parameter = request.getParameter(s: "name"); // Tham số trong URL hoặc form

        // 2. Quản lý phiên làm việc (Session)
        HttpSession session = request.getSession();
        session.setAttribute(s: "user", o: "GuestUser"); // Gán giá trị vào session
        String sessionId = session.getId(); // Lấy ID của session

        // 3. Truy cập header HTTP
        String userAgent = request.getHeader(s: "User-Agent"); // Lấy thông tin trình duyệt
        String contentType = request.getHeader(s: "Content-Type"); // Kiểu nội dung

        // 4. Xử lý dữ liệu từ form HTML (ví dụ với tham số "email")
        String email = request.getParameter(s: "email");
    }
}
```

localhost:8080/demo-request?name=John&email=john@example.com.

Thông tin HttpServletRequest

1. Thông tin yêu cầu HTTP:

Phương thức HTTP: GET

URL: http://localhost:8080/demo-request

Chuỗi truy vấn: name=John&email=john@example.com.

Tham số 'name': John

2. Thông tin phiên làm việc (Session):

ID phiên: 3AFBF44842CA774D0F23BEED982D8314

Giá trị 'user' trong session: GuestUser

3. Thông tin header HTTP:

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36

Content-Type: null

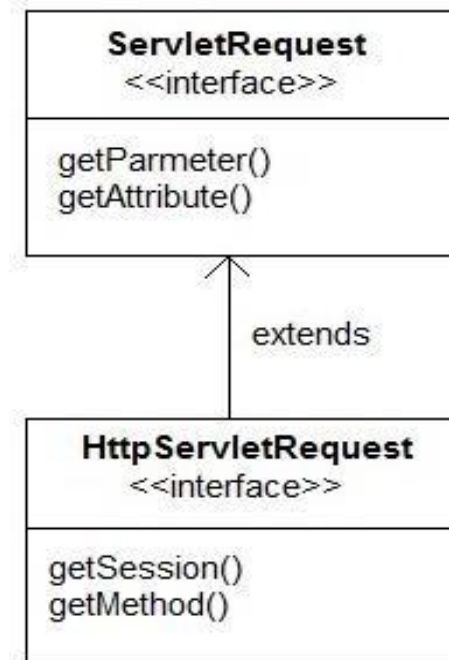
4. Dữ liệu từ form HTML:

Email: john@example.com.

04 Servlet Request and Response

❑ HttpServletRequ

String <code>getParameter(String name)</code>	returns value of parameter by name
void <code>removeAttribute(String name)</code>	removes an attribute from this request
void <code>setAttribute(String name, Object o)</code>	stores an attribute in this request.
String[] <code>getParameterValues(String name)</code>	returns an array of String objects containing all of the values the given request parameter has, or null if the parameter does not exist
Cookies <code>getCookies()</code>	returns an array containing all of the Cookie objects the client sent with this request
HttpSession <code>getSession()</code>	returns the current HttpSession associated with this request or, if there is no current session and create is true, returns a new session



04 HttpServletResponse

- **HttpServletResponse** là một giao diện (interface) trong Java Servlet API, được sử dụng để tạo và gửi phản hồi từ server tới client (thường là trình duyệt web).
- Nó cung cấp các phương thức để định nghĩa trạng thái HTTP, header, cookie, và nội dung phản hồi.

ServletResponse interface
(javax.servlet.ServletResponse)

```
<<interface>>
ServletResponse

getBufferSize()
setContentType()
getOutputStream()
getWriter()
setContentLength()
// MANY more methods...
```

HttpServletResponse interface
(javax.servlet.http.HttpServletResponse)

```
<<interface>>
HttpServletResponse

addCookie()
addHeader()
encodeURL()
sendError()
setStatus()
sendRedirect()
// MANY more methods...
```

04

Chức năng của HttpServletResponse

- **Thiết lập trạng thái HTTP:**
 - Đặt mã trạng thái HTTP (HTTP status code), ví dụ: 200 (OK), 404 (Not Found), 500 (Internal Server Error).
- **Thiết lập header HTTP:**
 - Thêm hoặc chỉnh sửa các header HTTP, như Content-Type, Content-Length, Cache-Control.
- **Gửi dữ liệu phản hồi:**
 - Gửi nội dung HTML, JSON, XML, hoặc bất kỳ loại dữ liệu nào khác tới client.
- **Chuyển hướng (Redirect):**
 - Chuyển hướng client tới một URL khác.
- **Làm việc với cookie:**
 - Thêm hoặc chỉnh sửa cookie để lưu thông tin trạng thái trên client.

04**Chức năng của HttpServletResponse**

```
@WebServlet("/response-demo") new *
public class HttpServletResponseDemo extends HttpServlet {
    @Override no usages new *
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // 1. Thiết lập trạng thái HTTP
        response.setStatus(HttpServletResponse.SC_OK); // Mã trạng thái 200 (OK)
        // 2. Thiết lập header HTTP
        response.setContentType("text/html");
        response.setCharacterEncoding("UTF-8");
        response.setHeader(s: "Custom-Header", s1: "DemoHeader");
        // 3. Gửi dữ liệu phản hồi
        response.getWriter().println("<html><body>");
        response.getWriter().println("<h1>HttpServletResponse Demo</h1>");
        response.getWriter().println("<p>Trạng thái: 200 OK</p>");
        response.getWriter().println("<p>Header 'Custom-Header': DemoHeader</p>");
        response.getWriter().println("</body></html>");
        // 4. Thêm cookie
        Cookie userCookie = new Cookie(name: "username", value: "JohnDoe");
        userCookie.setMaxAge(60 * 60); // Cookie tồn tại trong 1 giờ
        response.addCookie(userCookie);
        // 5. Chuyển hướng (nếu cần)
        // response.sendRedirect("https://www.example.com");
    }
}
```

**HttpServletResponse Demo**

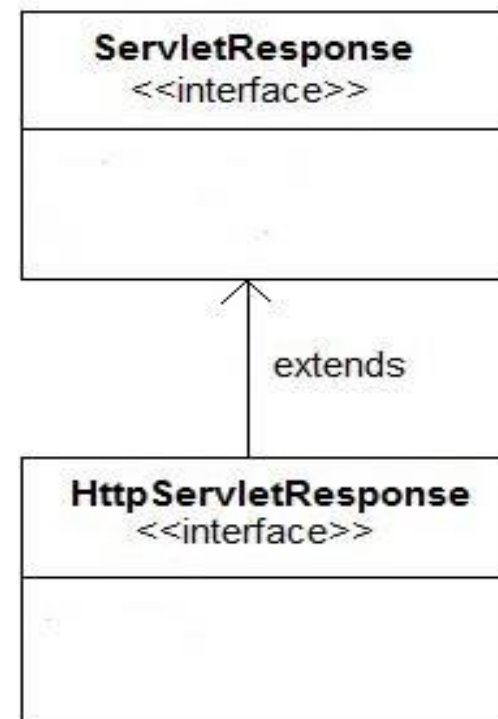
Trạng thái: 200 OK

Header 'Custom-Header': DemoHeader

04 **HttpServletResponse**

☐ Các method hay sử dụng

PrintWriter <code>getWriter()</code>	returns a PrintWriter object that can send character text to the client.
void <code>setContentType(String type)</code>	sets the content type of the response being sent to the client before sending the respond.
void <code>addCookie(Cookie cookie)</code>	adds the specified cookie to the response.
void <code>sendRedirect(String location)</code>	Sends a temporary redirect response to the client using the specified redirect location URL and clears the buffer



04 RequestDispatcher

- ☐ RequestDispatcher trong Java được sử dụng để chuyển tiếp (forward) yêu cầu từ một servlet đến một tài nguyên khác (như servlet, JSP, hoặc tệp tĩnh) hoặc để bao gồm (include) nội dung từ tài nguyên khác vào phản hồi.
- ☐ Nó thường được dùng trong các ứng dụng web để phân tách xử lý logic và hiển thị, hoặc để tái sử dụng các thành phần giao diện như header, footer.

04 RequestDispatcher

- ❑ Các phương thức chính của RequestDispatcher:
 - ❑ `forward(ServletRequest request, ServletResponse response)`:
 - Chuyển tiếp yêu cầu đến một tài nguyên khác. Servlet gốc sẽ dừng xử lý, và tài nguyên mới sẽ tiếp quản hoàn toàn việc xử lý yêu cầu.
 - ❑ `include(ServletRequest request, ServletResponse response)`:
 - Bao gồm nội dung của một tài nguyên khác vào phản hồi.
 - Cả tài nguyên gốc và tài nguyên được bao gồm đều đóng góp vào nội dung phản hồi.

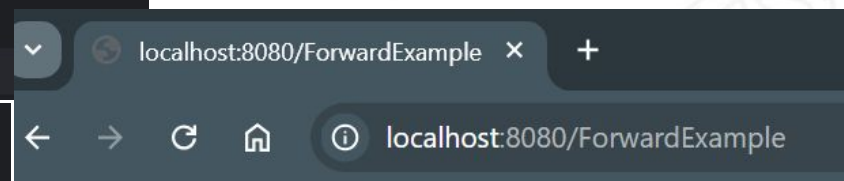
04 RequestDispatcher

- ❑ Các phương thức chính của RequestDispatcher:
 - ❑ `forward(ServletRequest request, ServletResponse response)`:
 - Chuyển tiếp yêu cầu đến một tài nguyên khác. Servlet gốc sẽ dừng xử lý, và tài nguyên mới sẽ tiếp quản hoàn toàn việc xử lý yêu cầu.
 - ❑ `include(ServletRequest request, ServletResponse response)`:
 - Bao gồm nội dung của một tài nguyên khác vào phản hồi.
 - Cả tài nguyên gốc và tài nguyên được bao gồm đều đóng góp vào nội dung phản hồi.

04 RequestDispatcher

```
@WebServlet("/ForwardExample") new *
public class ForwardExample extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response) no usages
        throws ServletException, IOException {
        // Thiết lập một thuộc tính để gửi đến tài nguyên khác
        request.setAttribute(s: "message", o: "Xin chào từ ForwardExample!");
        // Sử dụng RequestDispatcher để chuyển tiếp yêu cầu đến TargetServlet
        RequestDispatcher dispatcher = request.getRequestDispatcher(s: "/TargetServlet");
        dispatcher.forward(request, response);
    }
}
```

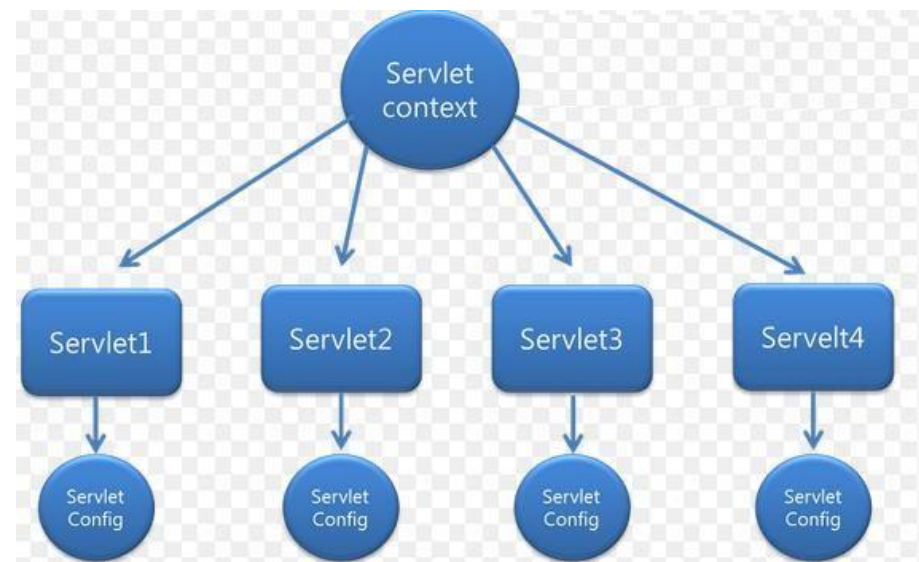
```
@WebServlet("/TargetServlet") new *
public class TargetServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Lấy thuộc tính được chuyển từ servlet ban đầu
        String message = (String) request.getAttribute(s: "message");
        response.setContentType("text/html");
        response.getWriter().println("<h1>" + message + "</h1>");
    }
}
```

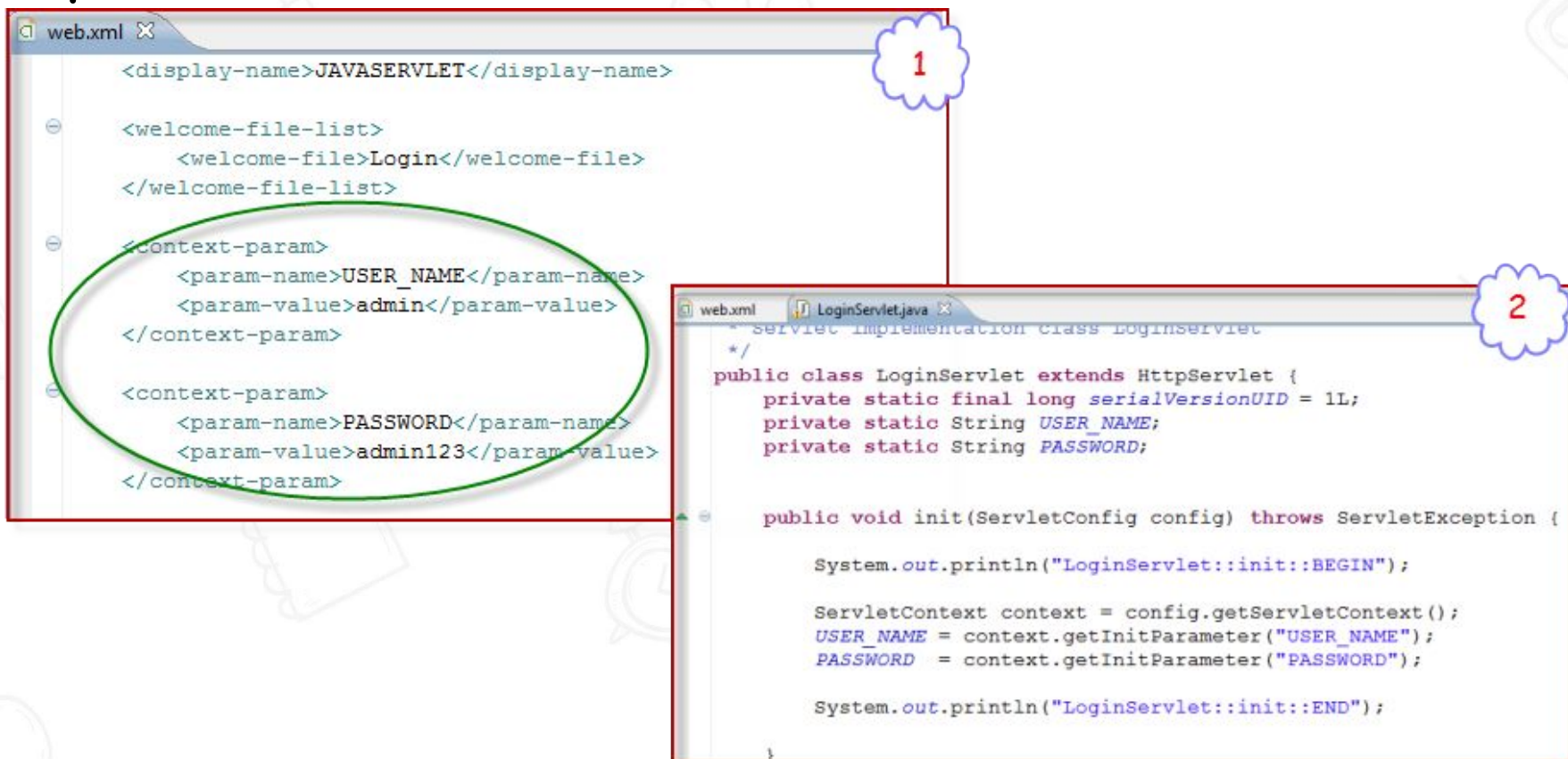


Xin chào từ ForwardExample!

04 ServletContext

- ❑ ServletContext trong java Servlet dùng để lưu các thông tin chung cho toàn bộ website. Mọi servlet có thể truy xuất thông tin từ đối tượng ServletContext.
- ❑ Bao gồm 3 phương thức chính
 - ❖ GetAttribute ()
 - ❖ SetAttribute ()
 - ❖ RemoveAttribute ()



04**ServletContext** **Ví dụ.**

1

```
<web.xml>
<display-name>JAVASERVLET</display-name>

<welcome-file-list>
  <welcome-file>Login</welcome-file>
</welcome-file-list>

<context-param>
  <param-name>USER_NAME</param-name>
  <param-value>admin</param-value>
</context-param>

<context-param>
  <param-name>PASSWORD</param-name>
  <param-value>admin123</param-value>
</context-param>
</web.xml>
```

2

```
Servlet Implementation Class LoginServlet
*/
public class LoginServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    private static String USER_NAME;
    private static String PASSWORD;

    public void init(ServletConfig config) throws ServletException {

        System.out.println("LoginServlet::init::BEGIN");

        ServletContext context = config.getServletContext();
        USER_NAME = context.getInitParameter("USER_NAME");
        PASSWORD = context.getInitParameter("PASSWORD");

        System.out.println("LoginServlet::init::END");
    }
}
```


04 ServletContext

- ❑ Cấu hình context parameters.

```
<context-param>
  <param-name>DB_URL</param-name>
  <param-value>jdbc:mysql://localhost:3306/test</param-value>
</context-param>

<context-param>
  <param-name>DB_Driver</param-name>
  <param-value>com.mysql.jdbc.Driver</param-value>
</context-param>
```

- ❑ Sau đó trong hàm init() có thể lấy tham số

```
// set up database connection and prepare SQL statements
public void init( ServletConfig config ) throws ServletException
{
    String dbDriver, dbURL;
    ServletContext context = config.getServletContext();

    dbDriver = context.getInitParameter("DB_Driver");
    dbURL = context.getInitParameter("DB_URL");

    // attempt database connection and create PreparedStatements
    // ...
}
```


Q & A